

به نام خدا



دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده برق و کامپیوتر



درس شبکه‌های عصبی و یادگیری عمیق

مدرس: دکتر احمد کلهر

تمرین شماره ۳

اردیبهشت ماه ۱۴۰۵

۳	تشخیص اشیاء به کمک روش های مبتنی بر ناحیه.....
۴	بخش اول: سوالات تئوری
۴	زیربخش اول: (مدل های دو مرحله ای) (۱۰ نمره).....
۴	زیربخش دوم: (مقایسه Fast R-CNN و Faster R-CNN) (۱۰ نمره).....
۴	زیربخش سوم: (معماری Faster R-CNN) (۲۰ نمره).....
۵	زیر بخش چهارم: (معماری Sparse r-cnn) (۱۵ نمره).....
۶	بخش دوم: سوالات پیاده سازی
۶	زیربخش اول: (آماده سازی داده مصنوعی) (۵ نمره).....
۷	زیربخش دوم: (تقسیم بندی داده ها و ساخت dataloader) (۵ نمره).....
۷	زیربخش سوم: (پیاده سازی معماری Sparse r-cnn) (۱۵ نمره).....
۸	زیربخش چهارم: تعریف تابع هزینه و تطبیق دوطرفه (bipartite matching) (۱۰ نمره).....
۸	زیربخش پنجم : تعیین هایپرپارامتر ها و آموزش مدل (۵ نمره).....
۸	زیربخش ششم : ارزیابی نهایی و تحلیل (۱۵ نمره).....
۱۰	نکات تحویل.....

تشخیص اشیاء به کمک روش های مبتنی بر ناحیه

تشخیص اشیا یکی از مهم ترین مسائل حوزه ی بینایی ماشین است که هدف آن شناسایی و مکان یابی اشیای موجود در تصویر یا ویدئو می باشد. در این مسئله ، مدل باید علاوه بر تشخیص نوع هر شیء ، محل قرارگیری آن را نیز به صورت جعبه های مرزی (Bounding Boxes) مشخص کند. در روش های سنتی، معمولاً تعداد زیادی ناحیه ی پیشنهادی (Proposal) تولید می شود که باعث افزایش هزینه ی محاسباتی می گردد. در این تمرین، معماری Sparse R-CNN معرفی می شود که با استفاده از تعداد محدودی Proposal قابل یادگیری، فرآیند تشخیص را انجام می دهد.

برخلاف روش های مبتنی بر Anchor، در Sparse R-CNN ناحیه ها و جعبه های پیشنهادی و اجزایش به صورت مستقیم در طول آموزش یاد گرفته می شوند و به تدریج به سمت مکان واقعی اشیا بهبود پیدا می کنند. این فرآیند باعث می شود شبکه بتواند ارتباط بهتری میان ویژگی های تصویر و مکان اشیا برقرار کند. همچنین، مدل به صورت انتها به انتها (End-to-End) آموزش داده می شود.

در این پروژه قصد داریم پیاده سازی نسخه ساده شده ای از Sparse R-CNN را اجرا و بررسی کنیم و با مفاهیمی مانند Proposal های قابل یادگیری ، استخراج ویژگی ناحیه ای (ROI Align) ، و بهبود تدریجی جعبه های تشخیص آشنا شویم. همچنین عملکرد مدل و کیفیت نتایج به دست آمده مورد تحلیل قرار خواهد گرفت.

بخش اول: سوالات تئوری

زیربخش اول: (مدل های دو مرحله ای) (۱۰ نمره)

- چهار معماری معروف دو مرحله ای برای تشخیص اشیاء که در کلاس درس عنوان شد را نام ببرید.
- تفاوت های اصلی معماری های دو مرحله ای با یک مرحله ای چه هستند؟
- یک مزیت و یک عیب معماری های دو مرحله ای نسبت به یک مرحله ای را عنوان کنید و توضیح دهید با چه تفاوت هایی در معماری ها (و یا چه تکنیک هایی) این مزیت و عیب به وجود آمده است.

زیربخش دوم: (مقایسه Fast R-CNN و Faster R-CNN) (۱۰ نمره)

- توضیح مختصری راجع به معماری این دو مدل از ورودی تا خروجی (مرحله به مرحله) بدهید.
- دو مزیت Faster R-CNN نسبت به Fast R-CNN چیست؟ هر کدام از این مزیت ها با ایجاد و یا تغییر چه چیزی در معماری به وجود آمده اند؟

زیربخش سوم: (معماری Faster R-CNN) (۲۰ نمره)

- نقش anchor ها در مدل Faster R-CNN چه هستند و چگونه در این معماری ایفای نقش میکنند؟
- در کلاس درس عنوانی به نام non-maximum-suppression(NMS) را یاد گرفتید ، درباره این روش توضیح بدهید.
- چرا در Faster R-CNN ها لازم هست که از تکنیک NMS استفاده شود؟
- چرا میتوان گفت در این معماری anchor ها dense (متراکم) هستند و نه sparse (پراکنده)؟ و این dense بودن چه مشکلی را به وجود می آورد؟

زیر بخش چهارم: (معماری Sparse R-CNN) (۱۵ نمره)

مقاله پیوست شده تحت عنوان (Sun et al., ۲۰۲۱) Sparse R-CNN را مطالعه کنید و به سوالات زیر پاسخ دهید .

- کلمه “sparse” در این مدل دقیقا به چه چیزی اشاره میکند ؟
- این مدل ، چه چیزهایی را نسبت به مدل Faster R-CNN حذف میکند و چه چیز هایی را اضافه میکند (سه مورد از تغییرات) ؟ هر کدام از این تغییرات چه تاثیری میگذارد ؟
- این معماری به جای RPN که مربوط به Faster R-CNN هست، چه راهکاری را انتخاب کرده است ؟
- آیا این معماری هنوز نیاز به NMS دارد ؟ توضیح دهید .

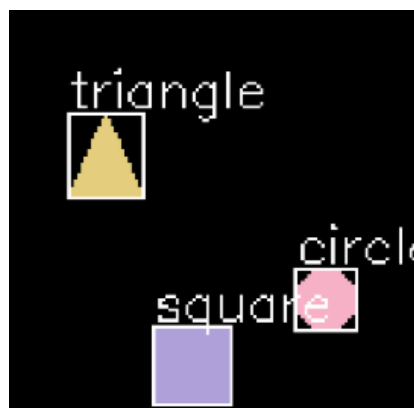
بخش دوم: سوالات پیاده سازی

در این بخش قصد داریم از معماری معرفی شده در مقاله پیوست شده (Sparse R-CNN) استفاده کنیم تا task تشخیص اشیاء (object detection) را با پیاده سازی های مربوطه انجام دهیم. با توجه به شرایط فعلی، پیاده سازی به گونه ای در نظر گرفته شده که بتوان کد ها را با CPU لپتاپ معمولی انجام داد. از آنجایی که پیاده سازی وظیفه object detection بر روی تصاویر واقعی نیازمند منابع پردازشی زیادی هست، داده های مخصوص آموزش را خودتان به صورت مصنوعی خواهید ساخت. دقت کنید که تحویل دادن کد ها برای هر بخش کافی نیست و باید گزارش کار کامل همراه با تحلیل و بررسی نتایج آورده شود.

زیربخش اول: (آماده سازی داده مصنوعی) (۵ نمره)

در این بخش قصد داریم خودمان مجموعه داده مصنوعی و toy شامل شکل های مختلف بسازیم تا در بخش های بعد، پیاده سازی تشخیص این اشکال را با کمک مدل معرفی شده انجام دهیم:

- یک dataset شامل ۱۰۰۰ تصویر 128×128 با پس زمینه مشکی درست کنید به نحوی که هر تصویر به صورت رندوم شامل یک تا سه شکل از بین اشکال مربع، مثلث و دایره باشد.
- برچسب صحیح (ground-truth) برای bounding box های متعلق به هر شکل (در قالب x, y, w, h) و همچنین برچسب صحیح عدد متناظر با آن شکل (class label) را ایجاد کنید. (۱: دایره، ۲: مربع، ۳: مثلث؛ عدد ۰ برای پس زمینه)
- برای آشنایی با دیتاست، ۵ نمونه از دیتاستی که ساختید را به نمایش در بیاورید به نحوی که برچسب مربوط به bounding box و class label روی شکل ها مشخص باشند مانند نمونه زیر:



زیربخش دوم: (تقسیم بندی داده ها و ساخت dataloader) (۵ نمره)

- ۹۰۰ تا از داده ها را برای آموزش اختصاص دهید و ۱۰۰ تا را برای آزمون قرار دهید .
- سپس با batch_size به مقدار ۱۶ ، train_dataloader و test_dataloader را ایجاد کنید .
- یک تابع collate_fn سفارشی برای مدیریت دیکشنری های هدف نوشته شود و از پارامتر collate_fn درون dataloader استفاده شود تا مشکلی برای اندازه های متغیر و متفاوت نداشته باشیم .

زیربخش سوم: (پیاده سازی معماری Sparse R-CNN) (۱۵ نمره)

- شما باید نسخه ای سبک از Sparse R-CNN شامل یک مدل پایه (Backbone) و یک سر پویا (Dynamic Head) بسازید.
- Backbone: یک شبکه عصبی کانولوشنی ۳ لایه پیاده سازی کنید که نقشه ای از ویژگی ها با ۱۲۸ کانال و رزولوشن 16×16 خروجی دهد.
- پیشنهاد های یادگیرنده (learning proposals): ۲۰ جعبه پیشنهادی (learnable proposal boxes) و ۲۰ ویژگی پیشنهادی یادگیرنده (learnable proposal features) را به عنوان nn.Parameter تعریف کنید.
- Iterative Refinement Head: یک حلقه تکرار شونده ۳ مرحله ای پیاده سازی کنید. در هر مرحله:
 - ROI Align: ویژگی های فضایی (spatial features) را از نقشه ویژگی های مدل پایه ، با استفاده از مختصات جعبه های اصلاح شده در مرحله فعلی استخراج کنید.
 - Dynamic Interaction: اجازه دهید ویژگی های پیشنهادی با ویژگی های ROI استخراج شده تعامل داشته باشند.
 - Refinement: از لایه های خطی برای پیش بینی احتمال کلاس و تغییرات مختصات جهت به روز رسانی جعبه ها برای مرحله بعد استفاده کنید.
- برای احتیاط از منطق torch.min و torch.max درون حلقه استفاده کنید تا اطمینان حاصل شود جعبه ها همیشه معتبر باقی می ماند . ($x_1 < x_2$ و $y_1 < y_2$)

زیربخش چهارم: تعریف تابع هزینه و تطبیق دوطرفه (bipartite matching) (۱۰)

نمره)

- با استفاده از الگوریتم مجارستانی و با استفاده از :
scipy.optimize.linear_sum_assignment یک تطبیق دهنده (matcher) بسازید.
تابع هزینه باید شامل هزینه طبقه‌بندی و هزینه فاصله L1 بین box ها باشد.
این بخش اصلاً وارد محاسبات گرادیان و آپدیت وزن‌ها نمی‌شود اما بدون matcher ، مدل هنگام آموزش نمی‌داند برای کدام جعبه باید Loss حساب کند. هدف ما این است که هر پیشنهاد (Proposal) یاد بگیرد که یا دقیقاً یک شیء را انتخاب کند و یا پس‌زمینه باشد. یعنی در انتهای کار در حالت کاملاً ایده آل ، اگر ۳ شیء در تصویر داریم ، فقط ۳ پیشنهاد باید امتیاز بالا داشته باشند و بقیه صفر شوند.
- تابع هزینه نهایی (Loss Criterion): یک تابع هزینه ترکیبی پیاده‌سازی کنید:
Focal Loss: برای طبقه‌بندی، جهت مدیریت عدم تعادل بین پس‌زمینه و اشیاء.
Box Loss: استفاده از L1 Loss برای جریمه کردن خطاهای مختصات.

زیربخش پنجم: تعیین هایپرپارامترها و آموزش مدل (۵ نمره)

برای دستیابی به همگرایی خوب روی CPU :

- بهینه‌ساز: از Adam با نرخ یادگیری $10^{-5} * 5$ استفاده کنید.
- مدل را برای epoch ۴۰ با آموزش دهید.
- نمودار loss آموزش را رسم کنید.

زیربخش ششم: ارزیابی نهایی و تحلیل (۱۵ نمره)

- معیار mAP: معیار Mean Average Precision را روی داده‌های تست محاسبه کنید. (دقت کنید که اگر قبل از ارزیابی، پیش‌بینی‌ها را نرمالیزه (۰, ۱) کرده اید ، آنها را به ابعاد پیکسل تصویر $128 * 128$ برگردانید).

- تحلیل بصری:

۱. پیش‌بینی‌های نهایی را به همراه کلاس پیش‌بینی شده برای هر شکل ، روی ۳ تصویر تست نمایش دهید.

۲. جعبه‌های پیشنهادی یادگیرنده (learnable proposals) را قبل و بعد از آموزش، روی یک تصویر تست، ترسیم کنید و با هم مقایسه کنید . در مورد تغییراتی که برای جعبه ها اتفاق افتاده ، توضیح دهید .

(توصیه : دقت کنید که در مرحله ارزیابی و نمایش نتایج ، مدل شما همیشه تعداد ثابتی جعبه (مثلاً ۲۰ عدد) تولید می‌کند. بسیاری از این جعبه‌ها مربوط به پس‌زمینه هستند یا اطمینان مدل به آن‌ها بسیار پایین است. برای نمایش نتایج نهایی و محاسبه دقیق mAP، بهتر است از یک متغیر کمکی به نام keep استفاده کنید:

- حذف پس‌زمینه ($labels \neq 0$): طبق تعریف دیتاست، کلاس ۰ متعلق به پس‌زمینه است. ما فقط می‌خواهیم جعبه‌هایی را نگه داریم که مدل ادعا می‌کند یکی از اشکال هندسی (دایره، مربع یا مثلث) هستند.

- آستانه اطمینان ($max_scores > 0.3$): مدل برای هر جعبه یک امتیاز (Score) بین ۰ تا ۱ می‌دهد. جعبه‌هایی که امتیاز پایینی دارند معمولاً پیش‌بینی‌های اشتباه یا نویز هستند. با قرار دادن یک آستانه (مثلاً ۰.۳) ، فقط مطمئن‌ترین تشخیص‌ها را اعمال می‌کنیم.)

نکات تحویل

- مهلت ارسال این تمرین تا پایان روز "جمعه ۱ خرداد ماه" خواهد بود.
- این زمان قابل تمدید نیست و در صورت نیاز می‌توانید از گریس استفاده کنید.
- در نظر داشته باشید که حداکثر مهلت آپلود تمرین در سامانه تا ۷ روز پس مهلت تحویل است و پس از آن سامانه بسته خواهد شد.
- پیاده سازی با زبان برنامه نویسی پایتون باید باشد و کدهای شما باید قابل اجرا بوده و به همراه گزارش آپلود شوند.
- انجام این تمرین به صورت یک نفره می‌باشد.
- در صورت مشاهده هر گونه تشابه در گزارش کار یا کدهای پیاده‌سازی، این امر به منزله تقلب برای طرفین در نظر گرفته خواهد شد.
- استفاده از کدهای آماده بدون ذکر منبع و بدون تغییر به منزله تقلب خواهد بود و نمره تمرین شما صفر در نظر گرفته می‌شود.
- در صورت رعایت نکردن فرمت گزارش کار نمره گزارش به شما تعلق نخواهد گرفت.
- تحویل تمرین به صورت دستنویس قابل پذیرش نیست.
- تمامی تصاویر و جداول مورد استفاده در گزارش کار باید دارای توضیح (caption) و شماره باشند.
- بخش زیادی از نمره شما مربوط به گزارش کار و روند حل مسئله است.
- لطفا گزارش، فایل کدها و سایر ضمیمات مورد نیاز را با فرمت زیر در سامانه بارگذاری نمایید.
- HW3_[Lastname]_[StudentNumber].zip
- در صورت وجود سوال و یا ابهام می‌توانید از طریق رایانامه زیر با موضوع NNDL_HW3 با دستیاران آموزشی در ارتباط باشید:

○ دستیار تمرین

a.nematollahi@ut.ac.ir

با آرزوی سلامتی و موفقیت روزافزون